Day 2 — Cheatsheet

> Quick reference for Delta Lake Advanced, Optimization, Cost Management, and Security & Governance.

1. Change Data Feed (CDF)

```
-- Enable CDF on a table
ALTER TABLE my_table SET TBLPROPERTIES (delta.enableChangeDataFeed = true);

-- Read changes since version 5
SELECT * FROM table_changes('my_catalog.my_schema.my_table', 5);

-- Read changes between timestamps
SELECT * FROM table_changes('my_catalog.my_schema.my_table',
    '2026-01-01T00:00:00', '2026-01-02T00:00:00');
```

CDF columns: `_change_type` (insert, update_preimage, update_postimage, delete), `_commit_version`, `_commit_timestamp`

2. Advanced MERGE Patterns

```
-- Conditional MERGE with delete handling
MERGE INTO silver_customers t
USING (SELECT * FROM bronze_customers) s
ON t.customer_id = s.customer_id
WHEN MATCHED AND s.is_deleted = true THEN DELETE
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

SCD Type 2 with AUTO CDC:

```
CREATE FLOW silver_customers_scd2
AS AUTO CDC INTO silver_customers
FROM STREAM bronze_customers
KEYS (customer_id)
SEQUENCE BY ingestion_ts
STORED AS SCD TYPE 2;
-- Adds __START_AT, __END_AT columns automatically
```

3. OPTIMIZE & Compaction

```
-- Compact small files into larger ones
OPTIMIZE my_catalog.my_schema.my_table;

-- Check file metrics
DESCRIBE DETAIL my_catalog.my_schema.my_table;
-- numFiles, sizeInBytes
```

When to run: After many small writes (streaming, frequent appends). Recommended: scheduled nightly.

4. Z-ORDER

```
-- Co-locate data by high-cardinality columns for data skipping
OPTIMIZE my_table ZORDER BY (customer_id, order_date);
```

Best for: Columns frequently used in WHERE/JOIN clauses. Max 4 columns.

5. Liquid Clustering (Recommended for new tables)

```
-- Create table with Liquid Clustering
CREATE TABLE my_table
CLUSTER BY (customer_id, order_date)
AS SELECT * FROM source;

-- Alter existing table
ALTER TABLE my_table CLUSTER BY (customer_id, order_date);

-- Remove clustering
ALTER TABLE my_table CLUSTER BY NONE;
```

Advantages over Z-ORDER: Automatic, incremental, no need to schedule OPTIMIZE ZORDER jobs.

6. VACUUM

```
-- Remove old file versions (default: 7 days retention)
VACUUM my_table RETAIN 168 HOURS;

-- Check before running (dry run)
SET spark.databricks.delta.vacuum.logging.enabled = true;
```

WARNING: After VACUUM, time travel to versions older than retention period is impossible.

7. Partitioning

```
-- Partition by low-cardinality column
CREATE TABLE fact_orders
PARTITIONED BY (order_year)
AS SELECT *, YEAR(order_date) AS order_year FROM source;
```

Rules of thumb: - Each partition should have **≥1 GB** of data - Low cardinality only (year, month, country) — NOT customer_id - **Prefer Liquid Clustering** over partitioning for new tables

8. Query Plan Analysis

```
-- Analyze query execution plan
EXPLAIN FORMATTED SELECT * FROM my_table WHERE customer_id = '123';
```

```
# PySpark
df.explain("formatted")
```

Look for: Partition pruning, data skipping, broadcast joins, shuffle warnings.

9. Cost Drivers

Component	Cost Driver	Optimization
Compute (DBUs)	Time × cluster size	Right-size, Job Clusters, serverless
Storage	GB stored	VACUUM, avoid over-partitioning

Cost hierarchy: Job Cluster (~2x cheaper) < All-Purpose < Interactive

10. Cost by Medallion Layer

Layer	Storage	Compute	Tip
Bronze	Largest (raw)	Low (ingestion)	Auto Loader incremental
Silver	Medium	Medium (transforms)	MERGE, not full overwrite
Gold	Smallest	High per row (aggregations)	MATERIALIZED VIEW

11. Unity Catalog — Access Control

```
-- Grant read access
GRANT SELECT ON TABLE catalog.schema.table TO `analysts@company.com`;

-- Grant all on schema
GRANT ALL PRIVILEGES ON SCHEMA catalog.schema TO `engineers@company.com`;

-- Revoke access
REVOKE INSERT, UPDATE ON TABLE catalog.schema.table FROM `readonly@company.com`;
```

Privilege types: `SELECT`, `MODIFY`, `CREATE`, `USAGE`, `ALL PRIVILEGES`

12. Row-Level Security (Dynamic Views)

```
CREATE VIEW secure_orders AS
SELECT * FROM orders
WHERE region = current_user_attribute('region');
-- Each user sees only their region's data
```

13. Column Masking

```
-- Create masking function
CREATE FUNCTION mask_email(email STRING)
RETURNS STRING
RETURN CASE
  WHEN is_member('pii_readers') THEN email
  ELSE CONCAT(LEFT(email, 2), '***@***.com')
END;

-- Apply to table column
ALTER TABLE customers ALTER COLUMN email SET MASK mask_email;
```

14. Data Lineage & Audit

```
-- Table-level lineage
SELECT * FROM system.access.table_lineage
WHERE target_table_name = 'gold_orders';

-- Column-level lineage
SELECT * FROM system.access.column_lineage
WHERE target_table_name = 'gold_orders';

-- Audit logs
SELECT * FROM system.access.audit
WHERE action_name = 'getTable'
AND request_params.full_name_arg LIKE '%customers%';
```

Quick Decision Matrix

Need	Solution
Compact small files	`OPTIMIZE`
Speed up filtered queries	Z-ORDER or Liquid Clustering
Remove old versions	`VACUUM`
Track row-level changes	Change Data Feed (CDF)
SCD Type 2 automation	`AUTO CDC` in Lakeflow
Restrict row access	Dynamic Views + `current_user()`
Hide sensitive columns	Column masking functions
Monitor costs	`system.billing.usage`